

NUMERICAL ANALYSIS

GUIDED LAB 4

ROB TOMPKINS

Note, all the files associated with Guided Lab 4 are checked into the :

<https://github.com/chtompki/MathSpring2026/tree/main/NumericalAnalysis/GuidedLab4>

1. Given an initial value problem $y' = f(t, y)$ for $t \in [a, b]$ with initial condition $y(a) = \alpha$, create a program that implements RK4 to approximate the solution $w_i \approx y(t_i)$ with fixed step-size $h = (b - a)/N$:

$$\begin{aligned}k_1 &= hf(t_i, w_i) \\k_2 &= hf\left(t_i + \frac{h}{2}, w_i + \frac{k_1}{2}\right) \\k_3 &= hf\left(t_i + \frac{h}{2}, w_i + \frac{k_2}{2}\right) \\k_4 &= hf(t_i + h, w_i + k_3) \\w_{i+1} &= w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)\end{aligned}$$

Be sure to design your code to be able to handle systems of equations (e.g. $y \in \mathbb{R}^n$). You may find the Matlab tutorial found at this link helpful.

Solution. The classical fourth-order Runge–Kutta method can be implemented directly from the formulas above. To allow the same routine to handle both a scalar differential equation and a system of differential equations, the numerical solution is treated internally as a column vector. Thus, if

$$y(t) = \begin{bmatrix} y_1(t) \\ \vdots \\ y_n(t) \end{bmatrix},$$

then $f(t, y)$ returns an $n \times 1$ column vector.

For a fixed number of steps N , define

$$h = \frac{b - a}{N}$$

and

$$t_i = a + ih, \quad i = 0, 1, \dots, N.$$

Starting with

$$w_0 = \alpha,$$

the RK4 approximation is computed for $i = 0, \dots, N - 1$ using

$$\begin{aligned} k_1 &= hf(t_i, w_i), \\ k_2 &= hf\left(t_i + \frac{h}{2}, w_i + \frac{k_1}{2}\right), \\ k_3 &= hf\left(t_i + \frac{h}{2}, w_i + \frac{k_2}{2}\right), \\ k_4 &= hf(t_i + h, w_i + k_3), \end{aligned}$$

followed by

$$w_{i+1} = w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

The following Matlab function implements this algorithm.

```
function [t,w] = rk4(f,a,b,alpha,N)
%RK4 Classical fourth-order Runge-Kutta method.
%
% [t,w] = rk4(f,a,b,alpha,N)
%
% Approximates the solution of
%
%      y' = f(t,y),      a <= t <= b,
%      y(a) = alpha,
%
% using N equally spaced RK4 steps.

h = (b-a)/N;

% Convert the initial condition to a column vector.
alpha = alpha(:);
n = length(alpha);

% Construct the time mesh.
t = linspace(a,b,N+1).';
```

```

% Preallocate storage for the numerical solution.
% Each row contains one approximation.
w = zeros(N+1,n);
w(1,:) = alpha.';

for i = 1:N

    % Current approximation as a column vector.
    wi = w(i,:).';

    k1 = h*f(t(i),wi);

    k2 = h*f(t(i) + h/2, ...
              wi + k1/2);

    k3 = h*f(t(i) + h/2, ...
              wi + k2/2);

    k4 = h*f(t(i) + h, ...
              wi + k3);

    wi_next = wi + ...
              (k1 + 2*k2 + 2*k3 + k4)/6;

    w(i+1,:) = wi_next.';
end
end

```

For example, for the scalar initial value problem

$$y' = y - t^2 + 1, \quad y(0) = 0.5,$$

on $0 \leq t \leq 2$, the routine can be called using

```
f = @(t,y) y - t.^2 + 1;
```

```
[t,w] = rk4(f,0,2,0.5,10);
```

```

disp ([t,w])

%=====Outputs=====
% >> gl4prob1
% 0      0.5000
% 0.2000    0.8293
% 0.4000    1.2141
% 0.6000    1.6489
% 0.8000    2.1272
% 1.0000    2.6408
% 1.2000    3.1799
% 1.4000    3.7323
% 1.6000    4.2834
% 1.8000    4.8151
% 2.0000    5.3054

```

The same routine works without modification for a system. For example, consider

$$\begin{aligned} y_1' &= y_2, & y_1(0) &= 0, & y_2(0) &= 1. \\ y_2' &= -y_1, \end{aligned}$$

Writing the system in vector form gives

$$y' = \begin{bmatrix} y_2 \\ -y_1 \end{bmatrix}.$$

The corresponding Matlab code is

```

f = @(t,y) [y(2); -y(1)];

alpha = [0;1];

[t,w] = rk4(f,0,2*pi,alpha,100);

plot(t,w(:,1),'LineWidth',1.5)
hold on
plot(t,w(:,2),'LineWidth',1.5)
legend('y_1','y_2')
xlabel('t')

```

```
ylabel('Approximate solution')
grid on
```

This code yields the plot in Figure 1 below:

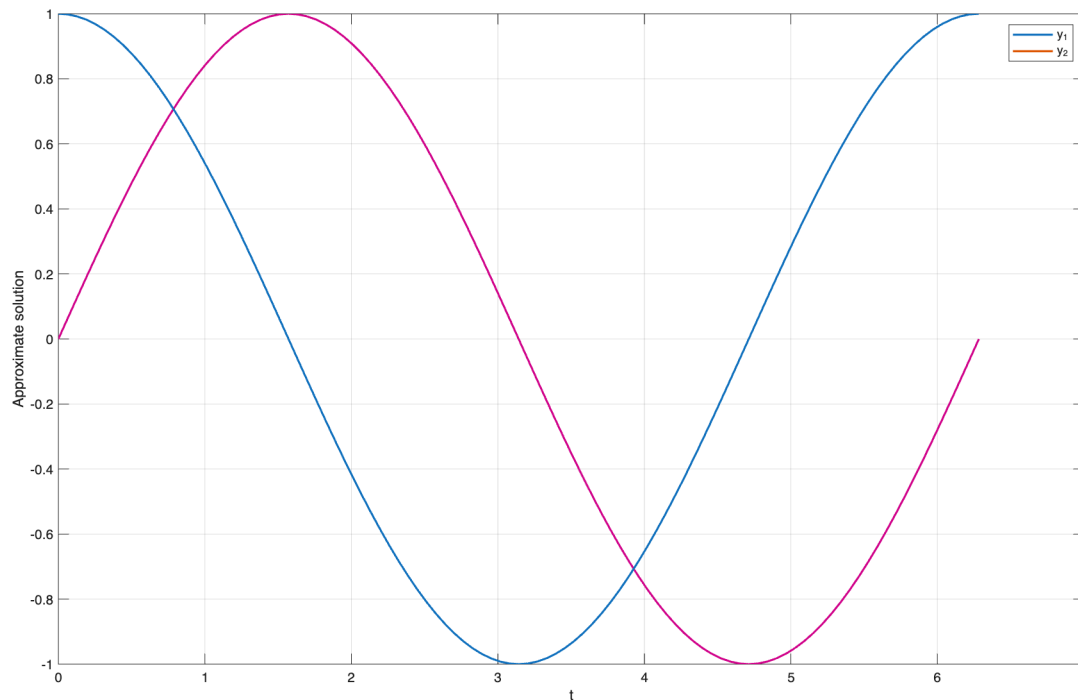


FIGURE 1. A solution for $y_1' = y_2$ and $y_2' = -y_1$.

Because w_i , k_1 , k_2 , k_3 , and k_4 may all be vectors, the RK4 formulas are unchanged when solving a system of equations. The classical RK4 method has global truncation error

$$O(h^4).$$

Consequently, in the asymptotic error regime, reducing the step size by a factor of two reduces the global discretization error by approximately $2^4 = 16$. $\square$

2. Consider the initial value problem

$$(1) \quad \frac{dy}{dt} = 1 - y + e^{2t}y^2 \quad (0 \leq t \leq 0.9), \quad y(0) = 0$$

- (1) Verify that the exact solution is $y(t) = e^{-t} \tan(e^t - 1)$.
- (2) Use your RK4 code to approximate the solution to the given initial value problem, using a stepsize of $h = 0.1$. Store the approximate solution at each time step as an element in a vector w . Also add a vector to your code which stores the errors, $err = x - w$.
- (3) Set up a script that solves the initial value problem for different step sizes $10^{-9} \leq h \leq 10^{-1}$ and stores the error at $t = 0.9$. Run this script on VCU's Athena cluster by submitting it as batch job (not interactively!). More information on how to do this can be found [here](#)). If you are using Matlab, you can find an example script for submitting your job [here](#). Make a log-log plot of the error vs. the stepsize and use this to verify the method converges with rate $O(h^4)$.

challenge: Assuming your code stores the solution at every step, you will likely become memory-limited if you try to test convergence using smaller step sizes. Modify your code to only save the final value of the solution, and go down to step sizes small enough to observe the effects of round-off error.

Solution. Note athena output files can be found at <https://github.com/chtompki/MathSpring2026/tree/main>

1. VERIFICATION OF THE EXACT SOLUTION

Let $u = e^t - 1$ and $y = e^{-t} \tan u$. Differentiating gives

$$\begin{aligned} y' &= -e^{-t} \tan u + e^{-t} \sec^2 u e^t \\ &= -y + 1 + \tan^2 u = 1 - y + e^{2t} y^2. \end{aligned}$$

Also $y(0) = \tan 0 = 0$, so

$$\boxed{y(t) = e^{-t} \tan(e^t - 1).}$$

The first positive pole occurs at $t_* = \log(1 + \pi/2) \approx 0.944216$, which lies beyond 0.9. Thus the solution is smooth throughout the requested interval. Since $f(t, y) = 1 - y + e^{2t}y^2$ is locally Lipschitz in y , this is the unique solution there.

2. CLASSICAL RK4 WITH $h = 0.1$

Set $t_n = nh$, $w_0 = 0$, and compute

$$\begin{aligned} k_1 &= f(t_n, w_n), & k_2 &= f(t_n + h/2, w_n + hk_1/2), \\ k_3 &= f(t_n + h/2, w_n + hk_2/2), & k_4 &= f(t_n + h, w_n + hk_3), \\ w_{n+1} &= w_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

In MATLAB, entry $n + 1$ represents time t_n ; the vectors include the initial value. The code sets `x=exp(-t).*tan(expm1(t))` and `err=x-w`, preserving the requested signed errors.

t_n	$x_n = y(t_n)$	w_n	$x_n - w_n$
0.0	0.0000000000	0.0000000000	0.00000e+00
0.1	0.0955150033	0.0955152528	-2.49549e-07
0.2	0.1842903884	0.1842907307	-3.42349e-07
0.3	0.2703012310	0.2703013801	-1.49077e-07
0.4	0.3591134152	0.3591129732	4.42060e-07
0.5	0.4598646541	0.4598633897	1.26439e-06
0.6	0.5906730569	0.5906721448	9.12094e-07
0.7	0.7972935784	0.7972918936	1.68477e-06
0.8	1.2493126185	1.2487983913	5.14227e-04
0.9	3.6413433579	3.4902224437	1.51121e-01

In particular, $y(0.9) \approx 3.64134335794140$ and $w_9 \approx 3.49022244367240$, with signed error 0.15112091426900. The relatively large error at this coarse step size reflects the rapid growth near the pole.

3. CONVERGENCE EXPERIMENT AND INTERPRETATION

The supplied script `rk4_ivp_solution.m` uses

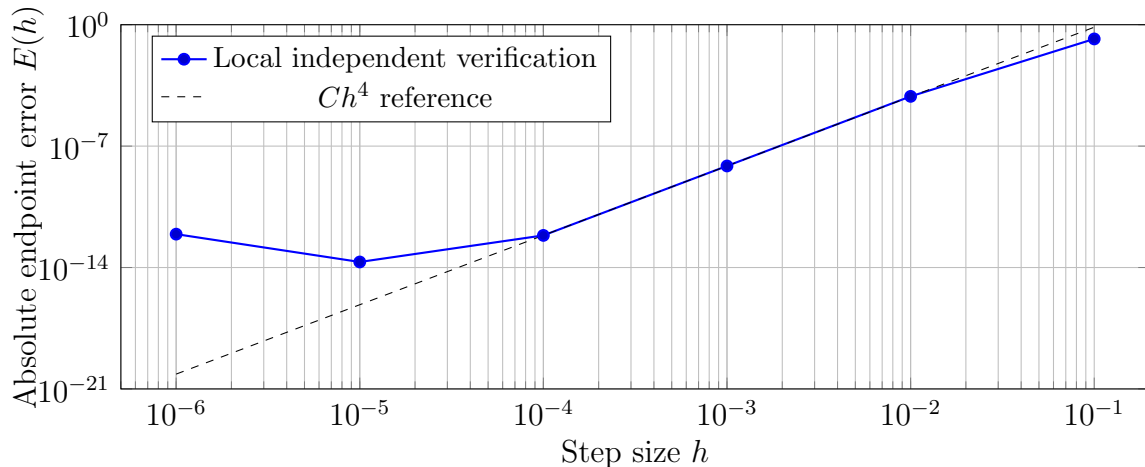
$$h_j = 10^{-j}, \quad N_j = 9 \cdot 10^{j-1}, \quad j = 1, \dots, 9, \quad E(h) = |y(0.9) - w_N|.$$

Internally it sets $h = 0.9/N$ and computes time from $t_n = nh$, avoiding repeated floating-point addition of the time variable. Successive observed orders are

$$p_j = \frac{\log(E(h_{j-1})/E(h_j))}{\log(h_{j-1}/h_j)}.$$

Verification status. The following table and plot were computed locally with an independent Python implementation of the same RK4 formulas in double precision. MATLAB and an authenticated Athena session were not available for this preparation. These are not Athena job results. The supplied MATLAB program performs the full requested sweep through 10^{-9} and writes its own tables, timing data, and plot.

h	Steps N	$E(h)$	Observed order
10^{-1}	9	1.511209e-01	—
10^{-2}	90	7.370413e-05	3.3118
10^{-3}	900	7.161237e-09	4.0125
10^{-4}	9,000	7.100986e-13	4.0037
10^{-5}	90,000	2.087219e-14	1.5318
10^{-6}	900,000	8.397727e-13	-1.6046



The orders 4.0125 and 4.0037 confirm fourth-order behavior in the truncation-dominated range. RK4 has one-step defect $O(h^5)$, which gives global error $O(h^4)$ on this fixed smooth interval. The coarse step $h = 0.1$ is not yet in that asymptotic range. A slope fitted over the

entire range, including round-off-dominated points, would not estimate the method's order reliably.

4. CHALLENGE: CONSTANT MEMORY AND ROUND-OFF

Storing one double-precision vector at $h = 10^{-9}$ requires approximately

$$8(N + 1) = 8(900,000,001) \text{ bytes} \approx 7.2 \text{ GB.}$$

Four full vectors (t, x, w, err) require about 28.8 GB before temporary arrays and MATLAB overhead. The convergence solver instead keeps only the current scalar y , the scalar time and index, and four RK stages. Its working storage is $O(1)$ in the number of steps. It uses a scalar `while` loop and never constructs a full grid for the fine runs. The ten-entry vectors required in part 2 are still retained.

Constant memory does not remove the computational cost: the finest run requires 9×10^8 steps, and all nine runs require 999,999,999 steps. Runtime remains $O(1/h)$. Results are saved after each completed step size so that a batch time limit does not erase earlier runs.

In exact arithmetic, the discretization error continues decreasing like h^4 . In floating-point arithmetic, rounding in stage evaluations and updates eventually dominates. Already in the local check, the error falls to approximately 2.1×10^{-14} at $h = 10^{-5}$ and rises to approximately 8.4×10^{-13} at $h = 10^{-6}$. The detailed values and turning point depend on the platform and arithmetic evaluation. The double-precision evaluation of the exact endpoint also limits the accuracy of the reported error near machine precision.

A useful qualitative model is

$$E(h) \lesssim C_T h^4 + C_R \frac{u}{h},$$

where u denotes unit round-off. The second term is a pessimistic accumulation model, not a universal measured law; less correlated round-off can grow more slowly. Perturbations can also be amplified by $f_y = -1 + 2e^{2t}y$ along this rapidly growing solution. The expected plot therefore decreases with slope approximately four before flattening or becoming irregular and increasing. Zero computed errors, if any, are omitted from the log-log plot and reported by the script.

5. RUNNING THE SUPPLIED FILES

For a short MATLAB check (down to 10^{-5}), run:

```
setenv('RK4_QUICK', '1');
rk4_ivp_solution
```

The full experiment is enabled by default. Submit it on Athena as a batch job using the included `athena_rk4.slurm`; do not execute the full experiment on a login node.

6. ATHENA BATCH SUBMISSION

Place `rk4_ivp_solution.m` and `athena_rk4.slurm` in the same writable directory on Athena. From that directory submit:

```
sbatch athena_rk4.slurm
squeue -u "$USER"
```

The batch file is:

```
#!/bin/bash
#SBATCH --job-name=rk4_ivp
#SBATCH --output=rk4_%j.out
#SBATCH --error=rk4_%j.err
#SBATCH --cpus-per-task=1
#SBATCH --mem=4G
#SBATCH --time=1-00:00:00

set -e
cd "$SLURM_SUBMIT_DIR"
module load matlab/R2024a
export RK4_QUICK=0
matlab -singleCompThread -batch "rk4_ivp_solution"
```

The one-day wall-time request is a starting allowance, not a runtime prediction; adjust it to the applicable queue limits and measured speed. Only one CPU is requested because each trajectory is advanced serially. The module and submission command follow VCU's MATLAB module instructions; also see the Athena cluster guide (documentation checked September 15, 2026).

The program writes `rk4_nodes.csv/.mat`, `rk4_convergence.csv/.mat`, and `rk4_convergence.pdf/.pr` alongside the MATLAB script. The convergence CSV contains the step size, step count, final value, signed and absolute errors, observed order, and elapsed seconds. Do not launch overlapping runs into the same output directory.

To finish the cluster-specific part of the assignment, retain the job ID and logs, verify that all nine rows through $h = 10^{-9}$ are present, and use the generated MATLAB plot in the submitted report. No cluster job submission or completion is claimed in this document. $\square$

3. You will now use your RK4 code to reproduce figure 7.21 in your textbook. Consider the model for cell production in a chemostat

$$\begin{aligned}\dot{s} &= 1 - s - \frac{msx}{a + s} \\ \dot{x} &= \frac{msx}{a + s} - x\end{aligned}$$

with $m = 16$ and $a = 0.25$.

- (1) Find the fixed points of the system and determine their stability.
- (2) Apply RK4 with step size $h = 0.05$ to approximate the solution to the initial value problem for $0 \leq t \leq 12$ with initial conditions $s(0) = 0.5$ and $x(0) = 0.02$. Plot $s(t)$ and $x(t)$ and explain how your results are consistent to your linear stability analysis.
- (3) Now repeat the previous part, this time using $h = 0.06$ to approximate the solution. Comment on the differences in your plots for $s(t)$ and $x(t)$.

Solution.

□

4. In order to explore absolute stability of RK4, consider the test problem $y' = \lambda y$ with $y(0) = 1$ and $0 \leq t \leq 10$.

- (1) Analytically apply RK4 to the test problem to show that

$$w_{i+1} = \left(1 + h\lambda + \frac{1}{2}(h\lambda)^2 + \frac{1}{6}(h\lambda)^3 + \frac{1}{24}(h\lambda)^4\right) w_i.$$

This allows us to define $Q(h\lambda) = 1 + h\lambda + \frac{1}{2}(h\lambda)^2 + \frac{1}{6}(h\lambda)^3 + \frac{1}{24}(h\lambda)^4$.

- (2) Now we will determine the stability region. Allow $h\lambda$ be a complex number $h\lambda = x + iy$ where $i = \sqrt{-1}$. Create a contour plot that shows the $|Q| = 1$ level set over the complex plane. If you are using Matlab, you may find the command `meshgrid` useful for creating a grid of points in the complex plane as follows:

```
% Define the grid for the complex plane (z = h*lambda)
x = linspace(-3, 3, 301);
y = linspace(-3, 3, 301);
[X, Y] = meshgrid(x, y);
Z = X + 1i*Y;
```

The Matlab command `contour()` can then be used to create the requested plot. Determine which region in the plot is the stability region.

- (3) With $\lambda = -55.31$ determine a bound on h that guarantees $h\lambda$ will lie in the stability region for this method. Numerically apply RK4 to the test problem for h values inside and outside of this stability region. How do the solution differ? How do these results inform the differences you noticed in parts 3(b) and 3(c)?

- (4) We will see what happens to numerical solutions with $h\lambda$ combinations that lie in both the region of stability and the right half of the complex plane, $\text{Re}(\lambda h) > 0$. To explore this, set $\lambda = 1 + 20i$ and $h = 0.1$. The point $h\lambda$ should now lie in both the right half of the plane and in the region of stability – plot this point and the region of stability on the same axes.

The solution with this λ is

$$e^{(1+20i)t} = e^t(\cos(20t) + i \sin(20t)).$$

Should the solution converge to zero as $t \rightarrow \infty$? Apply the RK4 to the test problem with these values of λ and h . Does the numerical method converge to zero? Should you use a $h\lambda$ that falls in or outside of the stability region for this λ ?

Solution.

□